

软件工程导论作业一：调研报告

潘腾凯 37220232203786

大语言模型（LLM）正以前所未有的深度和广度重塑软件行业，它既是强大的生产力工具，也带来了深刻的结构性挑战。对于软件从业者而言，理解其双重影响并制定清晰的应对策略，是未来职业发展的关键。

一、有利影响：生产力革命与创新赋能

大语言模型最直接、最积极的影响体现在软件开发全生命周期的效率提升和创新模式变革上。

1. 开发效率的指数级提升

以 GitHub Copilot、Claude Code、Cursor、Augment 等为代表的 AI 编程助手，已经成为许多开发者不可或缺的工具。它们显著提升了编码效率。

代码生成与补全：开发者只需通过自然语言注释或函数签名，模型就能快速生成功能代码、算法实现、API 调用等。这极大地减少了编写样板代码和通用逻辑所需的时间。例如，要实现一个文件读写并按特定格式解析的功能，过去需要查阅文档并手动编写数十行代码，现在可能只需一条清晰的注释即可生成。

辅助调试与测试：当遇到代码错误时，开发者可以将错误信息和相关代码片段交给大语言模型进行分析，模型通常能快速定位问题原因并给出修复建议。此外，它还能根据函数功能自动生成单元测试用例，提升代码的健壮性。

知识获取与学习：大语言模型改变了开发者解决问题的方式。过去依赖搜索引擎和技术论坛（如 Stack Overflow）寻找答案，现在可以直接向模型提问，获得更具针对性、更结构化的解决方案和代码示例，学习曲线变得更加平缓。

2. 软件开发门槛的降低

大语言模型的自然语言理解能力，使得非专业开发者也能参与到软件创造中来。产品经理、设计师或数据分析师可以通过自然语言描述需求，让模型生成原型脚本或数据处理工具，

实现了低代码/无代码开发的进一步延伸。这加速了跨领域创新的步伐。

3. 软件产品形态的革新

大语言模型不仅是开发工具，其本身也正在成为软件产品的核心功能。从智能客服、个人助手到企业知识库，内置大语言模型的应用层出不穷。例如，微软的 **Microsoft 365 Copilot** 将 AI 能力深度集成到 Office 全家桶中，使其能够自动生成文档、分析数据、总结邮件，这为软件产品创造了全新的价值维度和交互体验。

二、不利影响：质量隐患与职业冲击

在享受技术红利的同时，大语言模型带来的负面影响和潜在风险同样不容忽视。

1. 代码质量与安全风险

“代码幻觉”与逻辑谬误：大语言模型生成代码的原理是基于其庞大的训练数据进行概率推断，它并不真正“理解”代码的内在逻辑。因此，它可能生成看似正确但存在细微逻辑错误、性能陷阱或无法处理边界条件的代码。过度依赖而未经严格审查，会将这些隐患引入生产环境。

安全漏洞的扩散：模型的训练数据来源于公开的开源代码库，其中包含了大量存在安全漏洞的代码范例。模型在生成代码时，可能会无意中复现这些不安全的编码模式。已有研究表明，AI 生成的代码中安全漏洞的出现频率不低。

知识产权与合规问题：大语言模型生成的代码可能源于受特定开源协议（如 **GPL**）保护的项目。如果企业在商业产品中使用了这些代码而未遵循相应协议，将面临严重的法律合规风险。针对 **GitHub Copilot** 的集体诉讼案正是这一问题的集中体现。

2. 从业者技能的空心化

对于初级开发者而言，过度依赖 AI 工具完成基础编码任务，可能导致其核心编程能力、算法理解和系统设计思维得不到充分锻炼。长此以往，容易形成“知其然不知其所以然”的局面，基本功不扎实，向上发展路径受阻。

3. 对部分岗位的冲击与替代

重复性高、模式化强的编码任务，如前端页面布局、简单的 **CRUD** 接口编写、常规脚本撰写等，是目前大语言模型最擅长替代的领域。这意味着专注于此类任务的初级岗位和外包服务正面临被自动化工具挤压的风险。行业的岗位需求结构正在发生变化，对人才的要求从“代码实现”转向“价值创造”。

三、软件从业者的应对之道：从执行者到赋能者与思想者

面对这场技术变革，软件从业者与其被动担忧，不如主动求变，将挑战转化为机遇。

1. 拥抱工具，成为高效的“AI 协作者”

拒绝使用 **AI** 工具无异于在工业时代放弃机器。软件从业者应主动学习和掌握如何与大语言模型高效协作。

精通提示词工程：学习如何通过精准、清晰、结构化的自然语言指令，引导模型生成高质量、符合需求的代码。优秀的提问能力将成为新的核心技能。

将 AI 融入 workflow：在编码、重构、测试、文档撰写等各个环节，主动思考如何利用 **AI** 提效，将自己从繁琐的低价值劳动中解放出来，专注于更具创造性的工作。

2. 提升核心能力，巩固自己的技能壁垒

大语言模型目前难以替代的是人类的深度思考和复杂决策能力。从业者应着力提升以下核心能力：

系统设计与架构能力：从单一功能实现转向整体系统设计，深入理解软件架构、可扩展性、高可用性和性能优化。定义系统边界、模块交互和技术选型，是 **AI** 尚无法胜任的宏观工作。

复杂问题诊断与调试：在 **AI** 辅助失效的领域，尤其是在复杂的分布式系统、底层性能瓶颈或非典型业务逻辑错误面前，人类工程师的经验、直觉和深度分析能力无可替代。

代码审查与质量把控：角色定位从“代码生产者”转变为“代码质量的最终责任人”。

利用批判性思维审查 AI 生成的代码，识别其潜在的逻辑缺陷、安全风险和性能问题，成为保障软件质量的关键防线。

业务理解与产品思维：深入理解业务需求背后的商业逻辑，能够将技术方案与产品价值紧密结合。这种连接技术与市场的能力，是创造差异化优势的关键。

3. 转型价值定位，探索新的职业方向

技术的演进必然催生新的职业角色。从业者可以关注并探索与 AI 结合的新兴领域，如 AI 应用开发工程师、AI 系统集成专家、模型训练与微调工程师等，将自己的软件工程能力与 AI 技术栈相结合，开辟新的职业赛道。

结论

大语言模型不是软件工程师的终结者。它正在淘汰那些仅会重复性编码的“码农”，同时也在成就那些善于利用工具、具备深度思考能力和系统思维的“软件专家”。作为软件人，我们唯一的应对之道就是持续学习、主动适应，将重心从“如何实现”转向“做什么”和“为什么这么做”，最终成为驾驭 AI、创造更高价值的思考者和赋能者，扮演更倾向于“设计师”的角色。